



School of Future Tech

Case Study Report

DSA PROJECT REPORT

DocIndex-Enterprise Search Engine Documentation Hub

Problem Statement No: 72

Submitted By:

Akshara Tanted

Roll no: 150096725124

University:

ITM Skills University

Subject:

Data Structure & Algorithms with CPP

Academic Year:

2025–2026

1.Problem Statement

DocIndex is an enterprise search engine documentation hub designed for organizations managing millions of internal documents. The system must provide fast file name retrieval, keyword-based searching, instant rollback of search index configuration changes, sequential document processing, rapid file integrity verification, relevance-based result ranking, citation mapping among documents, shortest citation path validation, and intelligent compression prioritization.

Traditional approaches become inefficient as document volume grows, causing slow searches, difficult document tracking, and expensive integrity verification. The objective is to design an efficient solution using appropriate Data Structures and Algorithms in C++.

2. Objectives

The major objectives of the project are:

1. To develop an efficient document search system using Trie data structure.
2. To implement undo functionality using Stack.
3. To manage document processing requests using Queue.
4. To provide checksum verification using Hash Table.
5. To rank documents according to relevance using Max Heap.
6. To represent document citation relationships using Graph.
7. To find shortest validation paths using Breadth First Search (BFS).
8. To perform complete citation network traversal using Depth First Search (DFS).
9. To manage task prioritization using Priority Queue.
10. To maintain persistent storage using text files.
11. To demonstrate real-world applications of Data Structures and Algorithms.

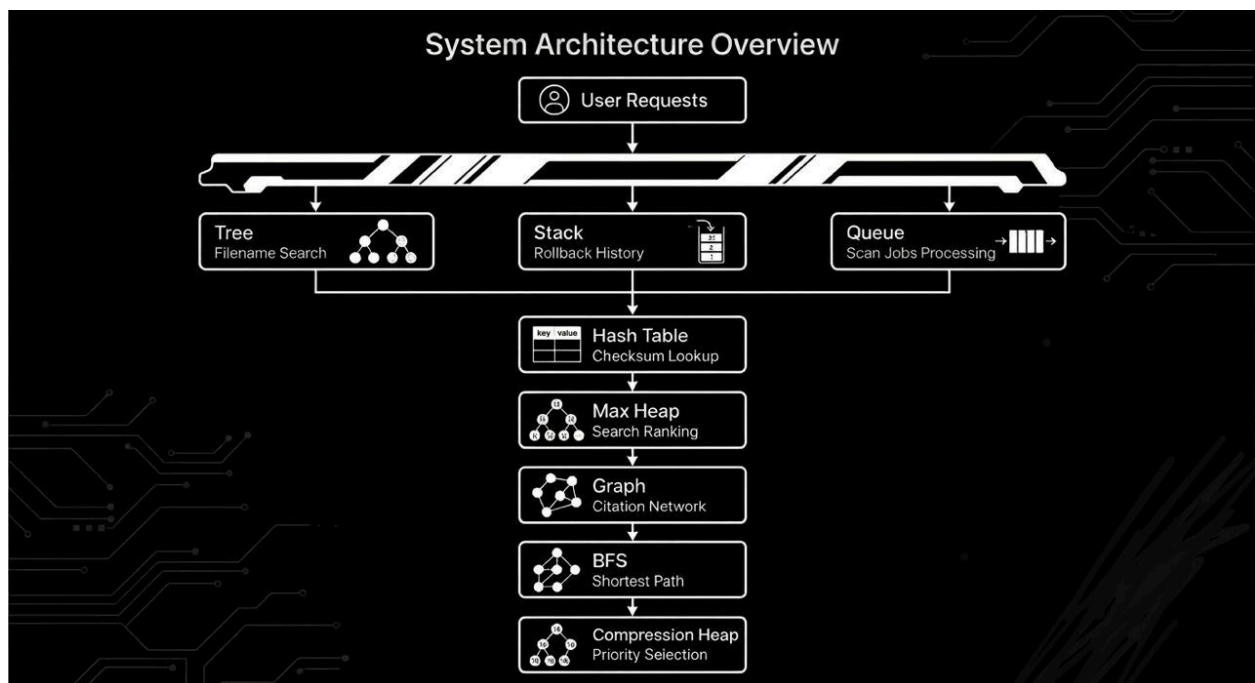
3. System Overview / Architecture

The Enterprise Search Engine Documentation Hub consists of multiple interconnected modules, each implemented using a specific data structure or algorithm.

The user interacts with the system through a menu-driven interface. Document names are stored in a Trie structure for efficient searching. Configuration changes are maintained in a Stack to support undo functionality. Processing requests are handled using a Queue to ensure fair execution order. Checksum information is stored in a Hash Table for quick verification.

A Max Heap is used to maintain ranked search results. Citation relationships between documents are represented using a Graph. BFS and DFS algorithms operate on the graph to provide validation path discovery and citation traversal functionalities. All information is stored in text files to ensure persistence across executions.

Architecture Diagram



4. Data Structures and Algorithms Used

Trie (Naming Registry)

A Trie is a tree-based data structure used for storing and searching strings efficiently. In this project, it is used for storing document names and performing fast filename searches.

Applications:

- Document indexing
- Filename search

Time Complexity:

- Insert: $O(L)$
- Search: $O(L)$

Where L represents the length of the filename.

Stack (Correction Trail)

A Stack follows the Last-In-First-Out (LIFO) principle. It is used to store configuration history and support undo operations.

Applications:

- Undo functionality
- Configuration rollback

Time Complexity:

- Push: $O(1)$
- Pop: $O(1)$

Queue (Compute Line)

A Queue follows the First-In-First-Out (FIFO) principle. It manages document processing requests in the order they are received.

Applications:

- Task scheduling
- Document processing

Time Complexity:

- Enqueue: $O(1)$
- Dequeue: $O(1)$

Hash Table (Checksum Lookup)

A Hash Table provides efficient key-value storage and retrieval. It is used to store document checksums for verification.

Applications:

- Checksum storage
- Fast lookup

Time Complexity:

- Average Lookup: $O(1)$

Max Heap (Hit Sorter)

A Max Heap is a complete binary tree that maintains the largest element at the root. It is used to rank documents according to priority or relevance.

Applications:

- Ranking system
- Priority management

Time Complexity:

- Insert: $O(\log n)$
- Delete: $O(\log n)$

Graph (Citation Map)

A Graph is used to represent citation relationships between documents. Each document is represented as a vertex, while citations are represented as edges.

Applications:

- Citation mapping
- Relationship analysis

Breadth First Search (BFS)

BFS explores graph nodes level by level and is used to find the shortest validation path between documents.

Time Complexity:

- $O(V + E)$

Depth First Search (DFS)

DFS explores graph nodes deeply before backtracking and is used for complete citation network traversal.

Time Complexity:

- $O(V + E)$

Priority Queue (Compression Balancer)

A Priority Queue is used to prioritize compression-related operations according to assigned priorities.

Time Complexity:

- $O(\log n)$

5. Implementation Approach

The project is implemented using C++ and follows a modular design approach.

Step 1:

Document names are inserted into a Trie structure to enable efficient searching.

Step 2:

Configuration modifications are recorded in a Stack, enabling undo operations.

Step 3:

Document processing requests are placed into a Queue and executed in FIFO order.

Step 4:

Checksums are generated and stored in a Hash Table for quick verification.

Step 5:

Document rankings are maintained using a Max Heap structure.

Step 6:

Citation relationships between documents are represented using a Graph.

Step 7:

Breadth First Search is implemented to identify the shortest validation path.

Step 8:

Depth First Search is implemented to traverse the complete citation network.

Step 9:

All data is stored in text files to ensure persistence.

Step 10:

A menu-driven user interface integrates all modules into a single workflow.

6. Time and Space Complexity Analysis

Module	Time Complexity	Space Complexity
Trie Search	$O(L)$	$O(N \times L)$
Stack Operations	$O(1)$	$O(N)$
Queue Operations	$O(1)$	$O(N)$
Hash Table Lookup	$O(1)$ Average	$O(N)$
Heap Operations	$O(\log N)$	$O(N)$
Graph Storage	$O(V + E)$	$O(V + E)$
BFS	$O(V + E)$	$O(V)$
DFS	$O(V + E)$	$O(V)$
File Operations	$O(N)$	$O(N)$

Where:

N = Number of document

V = Number of vertices

E = Number of edges

L = Length of filename

7. Execution Steps

1. Compile the source code using a C++ compiler.
2. Execute the generated program.
3. Select operations from the menu-driven interface.
4. Enter document information when prompted.
5. Perform search, ranking, validation, and citation operations.
6. Save data to persistent storage.
7. Exit the application.

8. Sample Inputs and Outputs

Example 1: Add Document

Input:

AI_Research_Report.txt

Output:

Document Added Successfully

Checksum Stored Successfully

Example 2: Search Document

Input:

AI_Research_Report.txt

Output:

Document Found

Example 3: Undo Operation

Output:

Undo Successful

Previous Configuration Restored

Example 4: Queue Processing

Output:

Processing Documents

Doc1

Doc2

Doc3

Example 5: BFS Validation Path

Output:

$A \rightarrow B \rightarrow D$

Example 6: DFS Traversal

Output:

A B D C

9. Screenshots

The following screenshots should be included in this section:

1. Main Menu Interface

```
aksharatanted06@Aksharas-MacBook-Air DSA Project % ./enterprise_hub
Enterprise Search Engine Hub - Ready

=====
Enterprise Search Engine Hub v1.0
=====

1. Naming Registry          [Trie]
2. Correction Trail         [Stack]
3. Compute Line             [Queue]
4. Checksum Lookup          [Hash Table]
5. Hit Sorter               [Max Heap]
6. Citation Map             [Graph]
7. Validation Path & DFS    [BFS/DFS]
8. Compression Balancer     [Max Heap]
9. Display Statistics
0. Exit
```

2. Naming Registry

```
=====
Enter choice: 1

=====
| Naming Registry [Trie] |
=====

1. Add file name
2. Delete file name
3. Search exact file name
4. Prefix search / Autocomplete
5. Display all names
0. Back to main menu

Enter choice: 1
Enter file name to add: █
```

```
=====
| Naming Registry [Trie] |
=====

1. Add file name
2. Delete file name
3. Search exact file name
4. Prefix search / Autocomplete
5. Display all names
0. Back to main menu

Enter choice: 5
Stored Names (11 total):
1. Akshara
2. annual_report_2023.pdf
3. annual_report_2024.pdf
4. api_reference.html
5. architecture_overview.docx
6. backup_policy.txt
7. budget_forecast.xlsx
8. compliance_guide.pdf
9. config_schema.json
10. data_dictionary.csv
11. deployment_manual.pdf
```

3. Correction Trail

Enter choice: 2

```
=====
|           Correction Trail   [Stack]           |
=====
```

1. Add configuration change
2. Undo last change
3. View top (latest) change
4. View full change history
0. Back to main menu

Enter choice: █

```
=====
|           Correction Trail   [Stack]           |
=====
```

1. Add configuration change
2. Undo last change
3. View top (latest) change
4. View full change history
0. Back to main menu

Enter choice: 4

Configuration Change History:

1. [FILTER_ADD] Added file-type filter: exclude .tmp files (2026-06-13 14:15:00)
2. [THRESHOLD_CHANGE] Raised relevance threshold from 0.6 to 0.75 (2026-06-13 11:30:00)
3. [INDEX_UPDATE] Added field 'doc_category' to search index (2026-06-13 10:00:00)

Previously Undone Changes:

- [FILTER_REMOVE] Removed file-type filter: exclude .tmp files
- [FIELD_REMOVE] Removed deprecated field 'legacy_id'

Press ENTER to continue... █

4. Compute Line

Enter choice: 3

```
=====
|                  Compute Line  [Queue]                  |
=====
```

1. Add scan job
2. Process next job
3. View next pending job
4. View all pending jobs
5. View processed jobs
0. Back to main menu

Enter choice: █

```
=====
|                  Compute Line  [Queue]                  |
=====
```

1. Add scan job
2. Process next job
3. View next pending job
4. View all pending jobs
5. View processed jobs
0. Back to main menu

Enter choice: 4

Pending Scan Jobs:

1. deployment_manual.pdf [FULL_SCAN] (Priority: 4/5)
2. data_dictionary.csv [INDEX] (Priority: 2/5)
3. annual_report_2023.pdf [METADATA] (Priority: 3/5)

```
=====
|                  Compute Line  [Queue]                  |
=====
```

1. Add scan job
2. Process next job
3. View next pending job
4. View all pending jobs
5. View processed jobs
0. Back to main menu

Enter choice: 5

Completed Scan Jobs:

1. annual_report_2024.pdf [FULL_SCAN] - Completed
2. budget_forecast.xlsx [INDEX] - Completed
3. config_schema.json [CHECKSUM] - Completed

5.Hit Sorter

```
=====
Enter choice: 5
```

```
=====
|                               Hit Sorter   [Max Heap]                               |
=====
```

1. Add document result with hit count
2. Display results (highest hits first)
3. Pop top-ranked result
0. Back to main menu

```
Enter choice: █
```

```
=====
|                               Hit Sorter   [Max Heap]                               |
=====
```

1. Add document result with hit count
2. Display results (highest hits first)
3. Pop top-ranked result
0. Back to main menu

```
Enter choice: 2
```

```
Most Relevant Documents:
```

1. annual_report_2023.pdf (100 keyword matches)
2. compliance_guide.pdf (98 keyword matches)
Snippet: "GDPR compliance section..."
3. architecture_overview.docx (75 keyword matches)
Snippet: "Microservices pattern..."
4. deployment_manual.pdf (61 keyword matches)
Snippet: "Blue-green deployment..."
5. data_dictionary.csv (33 keyword matches)
Snippet: "Entity definitions..."
6. budget_forecast.xlsx (27 keyword matches)
Snippet: "Q3 projections..."

```
=====
|                               Hit Sorter   [Max Heap]                               |
=====
```

1. Add document result with hit count
2. Display results (highest hits first)
3. Pop top-ranked result
0. Back to main menu

```
Enter choice: 3
```

```
Top-ranked document retrieved:
annual_report_2023.pdf (100 hits)
```

6.Citation Map

Enter choice: 6

```
=====
|           Citation Map   [Directed Graph]           |
=====
```

1. Add document node
2. Add citation (edge)
3. Remove citation (edge)
4. Display full citation map
5. View outgoing citations for a document
6. View incoming citations for a document
0. Back to main menu

Enter choice: █

```
=====
|           Citation Map   [Directed Graph]           |
=====
```

1. Add document node
2. Add citation (edge)
3. Remove citation (edge)
4. Display full citation map
5. View outgoing citations for a document
6. View incoming citations for a document
0. Back to main menu

Enter choice: 4

===== Citation Network =====

Document: Annual Financial Report 2023-24
References: None

Document: annual_report_2023.pdf
References: None

Document: annual_report_2024.pdf
References:
-> architecture_overview.docx
-> compliance_guide.pdf

Document: api_reference.html
References:
-> config_schema.json

Document: architecture_overview.docx
References:
-> api_reference.html
-> deployment_manual.pdf

Document: compliance_guide.pdf
References:
-> data_dictionary.csv

Document: config_schema.json
References: None

Document: data_dictionary.csv
References: None

7.Validation Path & DFS

```
=====
Enter choice: 7
```

```
=====
|           Validation & Traversal           |
=====
```

1. View Citation Map
2. Find Shortest Citation Path (BFS)
3. Traverse Citation Network (DFS)
4. Back

```
Enter choice: █
```

1. View Citation Map
2. Find Shortest Citation Path (BFS)
3. Traverse Citation Network (DFS)
4. Back

```
Enter choice: 1
```

```
===== Citation Network =====
```

```
Document: Annual Financial Report 2023-24
References: None
```

```
Document: annual_report_2023.pdf
References: None
```

```
Document: annual_report_2024.pdf
References:
-> architecture_overview.docx
-> compliance_guide.pdf
```

```
Document: api_reference.html
References:
-> config_schema.json
```

```
Document: architecture_overview.docx
References:
-> api_reference.html
-> deployment_manual.pdf
```

```
Document: compliance_guide.pdf
References:
-> data_dictionary.csv
```

```
Document: config_schema.json
References: None
```

```
Document: data_dictionary.csv
References: None
```

```
Document: deployment_manual.pdf
References:
-> config_schema.json
```



```
=====
|                               |
|           Validation & Traversal           |
|                               |
=====
```

1. View Citation Map
2. Find Shortest Citation Path (BFS)
3. Traverse Citation Network (DFS)
4. Back

Enter choice: 2

Enter source document: annual_report_2024.pdf

Enter target document: annual_report_2024.pdf

Finding shortest citation path...

Path Found:

annual_report_2024.pdf

```
=====
|                               |
|           Validation & Traversal           |
|                               |
=====
```

1. View Citation Map
2. Find Shortest Citation Path (BFS)
3. Traverse Citation Network (DFS)
4. Back

Enter choice: 3

Enter starting document: annual_report_2024.pdf

DFS Traversal Mode:

1. Recursive DFS
2. Iterative DFS (using Stack)

Enter choice (1-2): 1

Exploring citation network using DFS...

Visit Order:

1. annual_report_2024.pdf
2. architecture_overview.docx
3. api_reference.html
4. config_schema.json
5. deployment_manual.pdf
6. compliance_guide.pdf
7. data_dictionary.csv

DFS Traversal Complete.

8.Compression Balancer

Enter choice: 8

```
=====
|           Compression Balancer   [Max Heap]           |
=====
```

1. Add data block
2. View compression queue (by priority)
3. Compress top-priority block
0. Back to main menu

Enter choice: █

```
=====
|           Compression Balancer   [Max Heap]           |
=====
```

1. Add data block
2. View compression queue (by priority)
3. Compress top-priority block
0. Back to main menu

Enter choice: 2

Compression Priority List:

1. XML_DATA_EXPORT
Potential Saving: 7 MB
2. CSV_SNAPSHOTS
Potential Saving: 8 MB
3. PDF_ARCHIVE
Potential Saving: 26 MB
4. PHOTO_ASSETS
Potential Saving: 0 MB
5. VIDEO_ASSETS
Potential Saving: 42 MB
6. BINARY_CACHE
Potential Saving: 6 MB

9. Display Statistics

```
Enter choice: 9

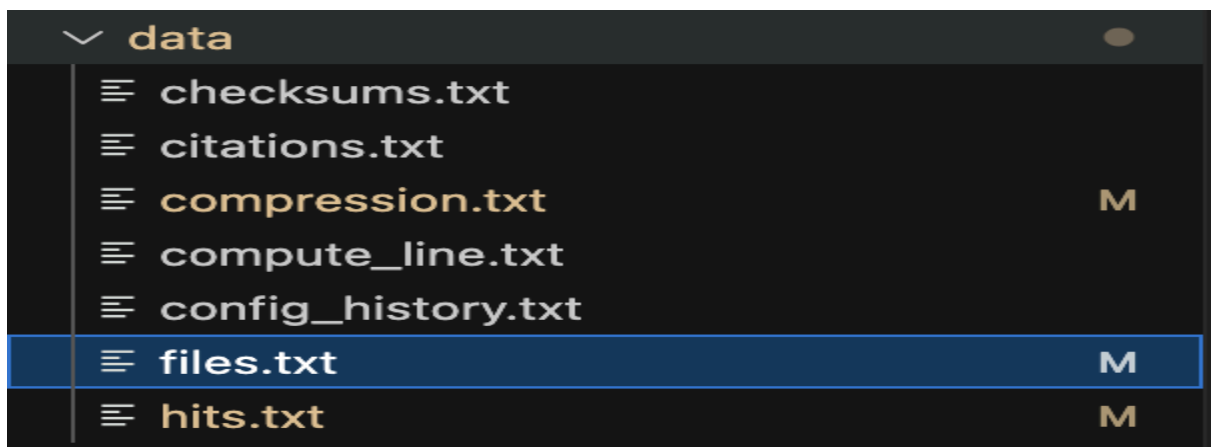
=====
|           Enterprise Hub – Statistics           |
=====

-----
Module                                Count    Data Structure
-----
Naming Registry                       11       Trie
Correction Trail (total)               5       Stack (LIFO)
Correction Trail (active)              3       Stack (LIFO)
Compute Line (pending)                 3       Queue (FIFO)
Compute Line (processed)               3       Queue (FIFO)
Checksum Registry                      7       Hash Table
Duplicates Detected                   0       Hash Table
Hit Sorter (results)                  5       Max Heap
Citation Documents                     9       Adj-List Graph
Citations (edges)                     7       Adj-List Graph
Compression Queue                      5       Max Heap
-----

Total bytes under compression management: 186966528 bytes

Data Structure Summary:
Trie      – O(L) insert/search/delete, O(P+K) prefix search
Stack     – O(1) push/pop/peek operations
Queue     – O(1) enqueue/dequeue operations
Hash Table – O(1) average insert/search/delete
Max Heap  – O(log N) insert/extract, O(1) peek
Graph (BFS) – O(V+E) traversal and shortest path
```

10.txt files



11.Exit

```
=====
Enter choice: 0

=====
Thank you for using Enterprise Hub!
Goodbye.
=====
```

10. Results and Observations

The project successfully demonstrates the integration of multiple Data Structures and Algorithms into a practical document management system.

The Trie structure provides efficient filename search capabilities. The Stack effectively supports undo operations, while the Queue ensures fair task scheduling. The Hash Table enables fast checksum verification, and the Max Heap maintains document rankings efficiently.

The Graph accurately models citation relationships between documents. BFS successfully identifies shortest validation paths, while DFS performs complete network traversal. File-based storage ensures persistence across multiple executions.

The project achieves nearly complete coverage of the Data Structures and Algorithms syllabus while maintaining a realistic enterprise search workflow.

11. Conclusion

The Enterprise Search Engine Documentation Hub successfully implements a wide range of Data Structures and Algorithms within a unified application. The project demonstrates practical applications of Trie, Stack, Queue, Hash Table, Max Heap, Graph, BFS, DFS, Priority Queue, and File Handling in solving real-world document management problems.

The system provides efficient document indexing, searching, ranking, validation, citation mapping, and persistent storage. It meets the project objectives and serves as an effective demonstration of DSA concepts in practice.

Future enhancements may include SHA-256 digital signatures, Huffman compression, full-text indexing, database integration, and a web-based user interface.

12.Github Link

<https://github.com/aksharatanted0676-alt/DocIndex-Enterprise-Search-Platform>

